

CSCE 606 SOFTWARE ENGINEERING · MODULE 10 · LECTURE NOTES

Continuous Integration and Delivery

Making release boring.

MODULE OUTCOMES

- Explain what CI actually requires beyond running tests.
- Design a pipeline with the right stages in the right order.
- Explain feature flags and why they enable trunk-based development.
- Compare deployment strategies and their rollback properties.
- Explain the DORA metrics and what they measure.

1 What continuous integration actually is

A PRACTICE, NOT A PRODUCT

'We have CI' almost always means 'we have a server that runs tests when a pull request opens'. That is a useful thing to have and it is not continuous integration.

Continuous integration is the practice of every developer integrating their work into a shared mainline at least daily. The automation supports it; it is not the thing itself.

A team with an excellent build server and three-week feature branches does not practise CI. They have automated testing and deferred integration pain — and the pain is deferred, not avoided.

The purpose is to discover integration problems while they are small and attributable to one small change. That requires integrating frequently; no amount of tooling substitutes.

Requirement	Why
A single mainline.	Multiple long-lived integration branches reproduce the problem CI exists to solve.
Everyone integrates at least daily.	This is the practice. Everything else is support.
Every integration triggers an automated build and test.	Otherwise integration problems are found later, by someone else.
A broken build is fixed immediately.	The highest priority for whoever broke it. See below.
The build is fast.	Under ten minutes. Beyond that, people stop waiting and start working around it.

A TOLERATED RED BUILD DESTROYS THE PRACTICE

If the build is frequently broken, people stop looking at it. A genuine failure becomes indistinguishable from the usual noise, which means the signal has zero value — not reduced value, zero.

Worse, teams develop the habit of merging onto a red build 'because it was already broken', at which point failures compound and nobody can tell which change caused what.

This is Module 05's flaky test problem at the scale of the whole pipeline, and the remedy is the same: **a red build stops other work until it is green**. Not 'should be looked at soon' — stops.

Teams that find this rule intolerable almost always have a build that is too slow or too flaky to sustain it. The correct response is to fix the build, not to relax the rule.

2 Pipeline design

Stage	Typical duration	What it catches
Lint, format, type check	Seconds	Mechanical problems. First, because it is cheapest.
Small tests (Module 05)	Under a minute	Logic errors.
Build artefacts	Minutes	Compilation and packaging failures.
Medium tests	Minutes	Component integration.
Large / end-to-end tests	Minutes	Whether the system actually works.
Deploy to staging	Minutes	Deployment mechanics and configuration.
Smoke tests	Seconds	Did it start and answer?

The ordering is an economic argument: each stage costs more than the one before, so fail at the earliest point possible. A type error should never consume twenty minutes of end-to-end test time before being reported.

TEN MINUTES IS A BEHAVIOURAL THRESHOLD, NOT A PREFERENCE

Below about ten minutes, a developer waits for the result and treats a failure as part of the current task.

Above it, they context-switch to something else. The failure now arrives when their attention is elsewhere, so it is deprioritised, so it sits broken longer, so others build on top of it.

A slow pipeline is not an inconvenience; it changes behaviour, and every behavioural change it causes is bad.

What to do: parallelise and shard the test suite; cache dependencies rather than resolving them each run; use build-system support for testing only what changed; move genuinely slow suites off the per-commit path to a nightly run.

3 Feature flags

SEPARATING DEPLOYMENT FROM RELEASE

Without flags, deploying code means releasing the feature in it. So incomplete work cannot be merged, so it lives on a branch, so branches live for weeks — which Module 08 identified as the source of merge difficulty.

With flags, you deploy code in a disabled state. The feature is released later by changing a configuration value, with no deployment at all. The two events are decoupled.

This single change unlocks a great deal: trunk-based development becomes possible for long-running work; features can be enabled for 1% of users and expanded; A/B experiments become configuration; and **rollback becomes turning a flag off**, which is faster and far safer than deploying a revert.

The cost is genuine. Each flag is a branch in the code, and n independent flags give 2^n configurations, only a few of which are ever tested. Flags must be removed once their decision is made.

Kind	Purpose	Intended lifetime
Release flag	Hide incomplete work so it can be merged.	Days to weeks, then deleted. These are the ones that accumulate and cause trouble.
Experiment flag	A/B testing.	The duration of the experiment, then deleted along with the losing branch.
Operational flag	Kill switch; shed load; disable an expensive feature under pressure.	Permanent, deliberately. These are infrastructure.
Permission flag	Entitlement by subscription tier or role.	Permanent. Arguably not a feature flag at all but part of the domain model, and better represented as such.

The practical discipline is to give every release flag an owner and an expiry date at creation, and to treat an expired flag as a build failure. Codebases where this is not enforced accumulate hundreds of flags, nobody knows which combinations are tested, and the flag system becomes the largest source of accidental complexity in the product.

4 Deployment strategies

Strategy	Mechanism	Rollback	Risk
Big bang	Stop the old version, deploy the new, start it.	Redeploy the previous version — minutes at best.	Downtime, and full exposure immediately.
Rolling	Replace instances in batches.	Roll the previous version back out, gradually.	Both versions run simultaneously, so they must be compatible.

Strategy	Mechanism	Rollback	Risk
Blue/green	Two complete environments; switch the load balancer.	Switch back. Seconds.	Twice the infrastructure; database compatibility across both.
Canary	Route a small percentage to the new version, watch metrics, expand gradually.	Stop and revert. Small blast radius throughout.	Requires good metrics and automated analysis to be worth it.

OPTIMISE FOR ROLLBACK SPEED

You cannot prevent every bad deployment. You can make undoing one cheap, and that is the more tractable problem.

A system that rolls back in thirty seconds tolerates a far higher deployment rate than one requiring an hour — and a higher deployment rate means smaller individual changes, which means fewer failures in the first place. The two reinforce each other.

Database migrations are the genuinely hard part, because many are not reversible: once a column is dropped, the data is gone. The discipline is **expand/contract**: add the new column; write to both old and new; backfill; switch reads to the new; and only then, in a later release, drop the old. Every step is independently deployable and independently reversible.

Never deploy a schema migration and the code that depends on it in the same release. That combination is what makes rollback impossible, and it is the usual cause of an incident becoming an outage.

5 Measuring delivery

Metric	Measures	Elite performance
Deployment frequency	How often code reaches production.	On demand — multiple times per day.
Lead time for changes	Commit to running in production.	Under one hour.
Change failure rate	Share of deployments causing degraded service.	Under 15%.
Time to restore service	How long recovery from an incident takes.	Under one hour.

SPEED AND STABILITY ARE POSITIVELY CORRELATED

The intuition is that deploying more often must mean breaking things more often, and that stability requires slowing down and adding review gates.

The DORA data consistently shows the opposite. Teams that deploy most frequently also have the lowest change failure rates and the fastest recovery. The two sets of metrics move together.

The mechanism is not mysterious. Frequent deployment forces small changes, and small changes are easier to review, easier to test, easier to diagnose when they fail, and easier to revert. Frequent deployment also means the deployment and rollback paths are exercised constantly, so they work when needed — Module 06's point about untested error paths, applied to operations.

A heavyweight change-approval process predicts worse outcomes on both axes, which is one of the more uncomfortable findings in the data and one of the most consistently ignored.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Martin Fowler — Continuous Integration	The article that defined the practice, and still the clearest statement that it is a practice rather than a tool.
Fowler — Feature Toggles	The taxonomy of §3 and the management discipline that keeps flags from becoming debt.
DORA — research and the four key metrics	The evidence base. Read the capability pages, not just the metric definitions.
Humble & Farley — Continuous Delivery	The standard reference for pipelines, deployment strategies, and database migration under continuous delivery.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Set up a CI pipeline for your Project 1 repository with the stages of §2, ordered by cost. Measure the total duration and the duration of each stage.
- 2 Make the pipeline faster. Parallelise the test suite, cache dependencies, and report before-and-after timings.
- 3 Implement a feature flag system, even a simple one. Deploy a disabled feature, then enable it by configuration without deploying.

- 4 Implement an expand/contract database migration: add a column, dual-write, backfill, switch reads, drop the old. Verify that you can roll back after each step.
- 5 Measure your own DORA metrics on a project: deployment frequency, lead time from commit to deploy, and how long a revert takes.
- 6 Deliberately break the build and measure how long it takes to notice and fix. Then make the pipeline report failures more visibly and repeat.
- 7 Find an open-source project with public CI. Examine its pipeline configuration and assess it against §2 — ordering, duration, and what runs per commit against nightly.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does continuous integration actually require, and why is a build server insufficient?
- 2 Why is a permanently red build worse than no CI at all?
- 3 What principle orders the pipeline stages?
- 4 Why is ten minutes a threshold rather than a preference?
- 5 What do feature flags decouple, and what does that make possible?
- 6 Name the four flag types and say which causes debt if not removed.
- 7 Why is rollback speed the right design criterion, and what makes database migrations hard?
- 8 What is the relationship between speed and stability in the DORA data, and why?